

DISCOVERY PHASE – COMPLETE (Zero assumptions; requirements locked verbatim from your query + full Sovereign Machina architecture + changelog v3.2.7 + latest repo state)

Core functionality required:

- Analyze the repository <https://github.com/Infisical/agent-vault> for deeper system and agent security implications.
- Evaluate how it can strengthen the current Sovereign Machina tri/quad-node stack (Node A = Mooncake KV buffer, Node B = Director, Node C = Oracle / Voice Core with Omi + Hermes, optional Hermes Pod).
- Focus on agent security (prompt injection, credential exfiltration), secrets management, auditability, and local-first design while preserving zero-trust, physical sovereignty, and declarative Nix identity.
- No changes to Node A or Node B core logic.

All requirements are now exhaustive and production-locked.

REPOSITORY SUMMARY

Agent-Vault is an open-source (MIT) **HTTP credential proxy and vault** designed specifically for AI agents. It acts as a secure intermediary: agents never see or store credentials. Instead, all HTTP traffic is routed through a local TLS proxy that injects secrets at the network layer.

Key Features:

- **Brokered Access:** Agents make normal HTTP calls; the proxy injects credentials transparently.
- **Zero Credential Exposure:** Prevents prompt injection or memory leaks from revealing secrets.
- **Encrypted Storage:** AES-256-GCM with random Data Encryption Key (DEK); optional Argon2id master password for rotation without re-encryption.
- **Audit Logging:** Per-request logs (method, host, path, status, latency, credential keys used) — no bodies/headers/query strings recorded.
- **Sandbox Support:** Container isolation (`--sandbox=container`) with egress lockdown and optional agent directory sharing.

- **Local-First:** Runs via CLI, Docker, or source; full local deployment with no cloud required.
- **SDK & CLI:** TypeScript SDK for sandboxed agents; simple `agent-vault run --claude` style usage.

Architecture:

- Proxy on port 14322 (TLS MITM).
- API/UI on port 14321.
- Sessions are scoped and short-lived.
- Works with any HTTP-speaking agent (Claude Code, Cursor, custom agents, etc.).

RELEVANCE & IMPLICATIONS FOR SOVEREIGN MACHINA SECURITY

1. Agent Security (Prompt Injection & Exfiltration Prevention)

This directly addresses one of the biggest risks in your current Hermes + GBrain setup: an agent (Hermes or subagent) could be tricked into leaking credentials via prompt injection or memory inspection. Agent-Vault ensures credentials never reach the agent's context or memory. All external calls (web fetch, Telegram, WhatsApp, computer APIs) can be routed through the proxy on Node C or the Hermes Pod. This is a **major security upgrade** for any external control surfaces.

2. Secrets Management & Rotation

- Credentials (API keys, tokens) are stored encrypted in a local vault on Node C/Hermes Pod.
- Master password support + DEK wrapping enables secure rotation without re-encrypting data.
- This complements your existing provenance tagging and Ouroboros auditing — every use is logged without exposing sensitive data.

3. Auditability & Compliance

- Per-request logs provide traceability (who/what/when) without logging payloads.

- This strengthens Ouroboros self-healing by giving Hermes a clean, immutable trail of external actions.

4. Local-First & Sovereignty Alignment

- Fully local deployment (CLI/Docker) fits your zero-cloud mandate.
- Sandboxing option adds isolation for risky agents (e.g., experimental subagents).
- No network egress except through the controlled proxy — perfect for your wired basement topology or VPN-extended phone use.

5. Integration Points in Current Stack

- **Best Placement:** Run Agent-Vault on the Hermes Pod (once available) or Node C. Hermes supervisor would create scoped sessions and route external calls through the proxy.
- **Voice/Screen Flow:** Omi voice notes or screen captures trigger intents → Hermes validates → external calls (if needed) go through Agent-Vault proxy.
- **GBrain Rewrite Synergy:** The rewritten GBrain (MemPalace + SQLite) can store vault metadata or session tokens securely, with Agent-Vault handling actual credential injection.
- **Machina Terminal:** The companion app can trigger actions that route through the proxy transparently.

6. Risks & Trade-offs

- **Added Complexity:** New service (proxy + vault) to manage on Node C/Hermes Pod.
- **Performance Overhead:** Minimal for most calls (local proxy), but monitor during high-volume web fetch.
- **Mitigation:** Start with a Nix module that runs Agent-Vault as a systemd service on the Hermes Pod with strict firewall rules (only allow traffic from local agents).

Overall Security Impact

This is a **high-value security layer** that meaningfully hardens agent behavior without compromising sovereignty. It directly mitigates credential exfiltration risks, adds auditable external control, and complements your existing Ouroboros/Hermes self-healing. It is a

natural fit for any web fetch, messaging (Telegram/WhatsApp), or computer automation extensions you plan.

ARCHITECT SIGN-OFF

Agent-Vault is a strong, production-ready addition for deeper system and agent security. It provides brokered credential access, encrypted storage, audit logging, and sandboxing that directly address prompt injection and exfiltration risks in your Hermes/GBrain setup. Running it on the Hermes Pod (or Node C for now) keeps the tri/quad-node stack clean and sovereign. The rewrite of GBrain in Rust/Go can easily integrate with it for secure, relational memory + controlled external actions. This elevates the overall security posture while preserving local-first design.

Your main sovereign machine remains completely untouched and rock-solid.

Clarifying questions (mandatory before next phase):

1. Do you want a concrete Nix module for running Agent-Vault on the Hermes Pod (or Node C) with sandboxing and MCP integration?
2. Shall I include a sample secure web fetch flow using Agent-Vault + Hermes?

Reply with your priority and I will deliver the concrete implementation. The security extension is ready to be added.